Rusmart Programs, SMT Outputs, and Challenges

Mehrad Haghshenas (21136163), WatIAM user ID: m3haghsh

April 2025

Acknowledgement

In writing the source code, *GitHub Copilot* and *ChatGPT* were used to assist in code generation mainly for syntax. The rest of this page is intentionally left blank.

Program 1

Program for testing Booleans:

Rust Program

```rust
// testing Boolean
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, SMT};

#[smt_impl(specs = _and_spec)]
fn _and(lhs: Boolean, rhs: Boolean) -> Boolean {
    lhs.and(rhs)
}

#[smt_spec(impls = _and)]
fn _and_spec(_lhs: Boolean, _rhs: Boolean) -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Boolean, rhs: Boolean) -> Boolean {
    _and_spec(lhs, rhs).eq(if *lhs { rhs } else { Boolean::from(false) })
}
```

SMT Output

```smt
; verification of impl-spec pair: _and ~> _and_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _and ((lhs Bool) (rhs Bool)) Bool
 (and lhs rhs)
)

(declare-fun _and_spec (Bool Bool) Bool)

; Define axioms:
(assert (forall ((lhs Bool) (rhs Bool)) (= (_and_spec lhs rhs) (ite lhs rhs false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Bool) (_rhs Bool) (lhs Bool) (rhs Bool)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_and lhs rhs) (_and_spec _lhs _rhs)))))
(declare-const _lhs Bool)
(declare-const _rhs Bool)
(declare-const lhs Bool)
(declare-const rhs Bool)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_and lhs rhs) (_and_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 2

Program for testing Booleans:

Rust Program

```rust
// testing Boolean
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, SMT};

#[smt_impl(specs = _and_spec)]
fn _and(lhs: Boolean, rhs: Boolean) -> Boolean {
    lhs.and(rhs)
}

#[smt_spec(impls = _and)]
fn _and_spec(_lhs: Boolean, _rhs: Boolean) -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Boolean, rhs: Boolean) -> Boolean {
    _and_spec(lhs, rhs).eq(if *lhs { Boolean::from(false) } else { rhs })
}
```

SMT Output

```smt
; verification of impl-spec pair: _and ~> _and_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _and ((lhs Bool) (rhs Bool)) Bool
 (and lhs rhs)
)

(declare-fun _and_spec (Bool Bool) Bool)

; Define axioms:
(assert (forall ((lhs Bool) (rhs Bool)) (= (_and_spec lhs rhs) (ite lhs false rhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Bool) (_rhs Bool) (lhs Bool) (rhs Bool)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_and lhs rhs) (_and_spec _lhs _rhs)))))
(declare-const _lhs Bool)
(declare-const _rhs Bool)
(declare-const lhs Bool)
(declare-const rhs Bool)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_and lhs rhs) (_and_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *sat - the specification does not correctly define the implementation.*

Program 3

Program for testing Booleans:

Rust Program

```rust
// testing Boolean
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, SMT};

#[smt_impl(specs = _and_spec)]
fn _and(lhs: Boolean, rhs: Boolean) -> Boolean {
    lhs.or(lhs.and(rhs.not().implies(rhs.xor(lhs))))
}

#[smt_spec(impls = _and)]
fn _and_spec(_lhs: Boolean, _rhs: Boolean) -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Boolean, rhs: Boolean) -> Boolean {
    _and_spec(lhs, rhs).eq(if *lhs {
        Boolean::from(true)
    } else {
        Boolean::from(false)
    })
}
```

SMT Output

```smt
; verification of impl-spec pair: _and ~> _and_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _and ((lhs Bool) (rhs Bool)) Bool
 (or lhs (and lhs (=> (not rhs) (or (and (not rhs) lhs) (and rhs (not lhs)))))))

(declare-fun _and_spec (Bool Bool) Bool)

; Define axioms:
(assert (forall ((lhs Bool) (rhs Bool)) (= (_and_spec lhs rhs) (ite lhs true false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Bool) (_rhs Bool) (lhs Bool) (rhs Bool)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_and lhs rhs) (_and_spec _lhs _rhs)))))
(declare-const _lhs Bool)
(declare-const _rhs Bool)
(declare-const lhs Bool)
(declare-const rhs Bool)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_and lhs rhs) (_and_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 4

Program for testing Integers:

Rust Program

```rust
// testing Integers
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_impl(specs = some_computation_spec)]
fn some_computation(hs: Integer) -> Integer {
    hs.mul(hs).div(hs)
}

#[smt_spec(impls = some_computation)]
fn some_computation_spec(_hs: Integer) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(hs: Integer) -> Boolean {
    some_computation_spec(hs).eq(hs)
}
```

SMT Output

```
; verification of impl-spec pair: some_computation ~> some_computation_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec some_computation ((hs Int)) Int
 (div (* hs hs) hs)
)

(declare-fun some_computation_spec (Int) Int)

; Define axioms:
(assert (forall ((hs Int)) (= (some_computation_spec hs) hs)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_hs Int) (hs Int)) (=> (= hs _hs) (= (some_computation hs) (some_computation_spec _hs)))))
(declare-const _hs Int)
(declare-const hs Int)
(assert (= hs _hs))
(assert (not (= (some_computation hs) (some_computation_spec _hs))))

(check-sat)
(exit)
```

Challenges: Theoretically, this should be unsat because zero cannot be an input, but this is *sat*. If we add the lines *(assert (distinct hs 0))* and *(assert (distinct _hs 0))* to the smt script, it will give *unsat*. This raises the question that do we want to do error handling like *attempt to divide by zero*? or assume that the user will not divide by zero? It is not just limited to this error, as other errors are not checked.

Output: *sat - the specification does not correctly define the implementation.*

Program 5

Program for testing Integers:

Rust Program

```rust
// testing Integers
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_impl(specs = some_computation_spec)]
fn some_computation(hs: Integer) -> Integer {
    hs.mul(hs).div(hs)
}

#[smt_spec(impls = some_computation)]
fn some_computation_spec(_hs: Integer) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(hs: Integer) -> Boolean {
    if *(hs.eq(Integer::from(0))) {
        some_computation_spec(hs).eq(Integer::from(0))
    } else {
        some_computation_spec(hs).eq(hs)
    }
}
```

SMT Output

```
; verification of impl-spec pair: some_computation ~> some_computation_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec some_computation ((hs Int)) Int
 (div (* hs hs) hs)
)

(declare-fun some_computation_spec (Int) Int)

; Define axioms:
(assert (forall ((hs Int)) (ite (= hs 0) (= (some_computation_spec hs) 0) (= (some_computation_spec hs) hs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_hs Int) (hs Int)) (=> (= hs _hs) (= (some_computation hs) (some_computation_spec _hs)))))
(declare-const _hs Int)
(declare-const hs Int)
(assert (= hs _hs))
(assert (not (= (some_computation hs) (some_computation_spec _hs))))

(check-sat)
(exit)
```

Challenges: Theoretically, this should be unsat because zero cannot be an input, but this is *uknown*. Writing

```
(define-fun-rec some_computation ((hs Int)) Int
 (ite (= hs 0) 0 (div (* hs hs) hs))
)
```

instead of the already defined function will give the expected *unsat*.

Output: *unknown - solver z3 cannot define whether the specification matches the implementation.*

Program 6

Program for testing Integers:

Rust Program

```rust
// testing Integers
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_impl(specs = some_computation_spec)]
fn some_computation(hs: Integer) -> Integer {
    hs.mul(hs).add(hs)
}

#[smt_spec(impls = some_computation)]
fn some_computation_spec(_hs: Integer) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(hs: Integer) -> Boolean {
    some_computation_spec(hs).eq(hs.mul(hs.add(Integer::from(1))))
}
```

SMT Output

```
; verification of impl-spec pair: some_computation ~> some_computation_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec some_computation ((hs Int)) Int
 (+ (* hs hs) hs)
)

(declare-fun some_computation_spec (Int) Int)

; Define axioms:
(assert (forall ((hs Int)) (= (some_computation_spec hs) (* hs (+ hs 1)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_hs Int) (hs Int)) (=> (= hs _hs) (= (some_computation hs) (some_computation_spec _hs)))))
(declare-const _hs Int)
(declare-const hs Int)
(assert (= hs _hs))
(assert (not (= (some_computation hs) (some_computation_spec _hs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 7

Program for testing Integers:

Rust Program

```rust
// testing Integers
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_impl(specs = some_computation_spec)]
fn some_computation(lhs: Integer, rhs: Integer) -> Boolean {
    lhs.add(rhs).gt(rhs)
}

#[smt_spec(impls = some_computation)]
fn some_computation_spec(_lhs: Integer, _rhs: Integer) -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Integer, rhs: Integer) -> Boolean {
    if *lhs.gt(Integer::from(0)) {
        some_computation_spec(lhs, rhs).eq(Boolean::from(true))
    } else {
        some_computation_spec(lhs, rhs).eq(Boolean::from(false))
    }
}
```

SMT Output

```
; verification of impl-spec pair: some_computation ~> some_computation_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec some_computation ((lhs Int) (rhs Int)) Bool
 (> (+ lhs rhs) rhs)
)

(declare-fun some_computation_spec (Int Int) Bool)

; Define axioms:
(assert (forall ((lhs Int) (rhs Int)) (ite (> lhs 0) (= (some_computation_spec lhs rhs) true) (= (some_computation_spec lhs rhs) false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Int) (_rhs Int) (lhs Int) (rhs Int)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (some_computation lhs rhs) (some_computation_spec _lhs _rhs)))))
(declare-const _lhs Int)
(declare-const _rhs Int)
(declare-const lhs Int)
(declare-const rhs Int)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (some_computation lhs rhs) (some_computation_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 8

Program for testing Reals:

Rust Program

```rust
// testing Rational
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT, Rational};

#[smt_impl(specs = some_computation_spec)]
fn some_computation(lhs: Rational, rhs: Rational) -> Boolean {
    lhs.add(rhs).gt(rhs)
}

#[smt_spec(impls = some_computation)]
fn some_computation_spec(_lhs: Rational, _rhs: Rational) -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Rational, rhs: Rational) -> Boolean {
    if *lhs.gt(Rational::from(0)) {
        some_computation_spec(lhs, rhs).eq(Boolean::from(true))
    } else {
        some_computation_spec(lhs, rhs).eq(Boolean::from(false))
    }
}
```

SMT Output

```smt2
; verification of impl-spec pair: some_computation ~> some_computation_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec some_computation ((lhs Real) (rhs Real)) Bool
 (> (+ lhs rhs) rhs)
)

(declare-fun some_computation_spec (Real Real) Bool)

; Define axioms:
(assert (forall ((lhs Real) (rhs Real)) (ite (> lhs (to_real 0)) (= (some_computation_spec lhs rhs) true) (= (some_computation_spec lhs rhs) false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Real) (_rhs Real) (lhs Real) (rhs Real)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (some_computation lhs rhs) (some_computation_spec _lhs _rhs)))))
(declare-const _lhs Real)
(declare-const _rhs Real)
(declare-const lhs Real)
(declare-const rhs Real)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (some_computation lhs rhs) (some_computation_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 9

Program for testing Reals:

Rust Program

```rust
// testing Rational
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, Rational, SMT};

#[smt_impl(specs = some_computation_spec)]
fn some_computation(hs: Rational) -> Rational {
    let x = Rational::from(-1);
    if *hs.ge(Rational::from(0)) {
        hs
    } else {
        hs.mul(x)
    }
}

#[smt_spec(impls = some_computation)]
fn some_computation_spec(hs: Rational) -> Rational {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(hs: Rational) -> Boolean {
    some_computation_spec(hs).ge(Rational::from(0))
}
```

SMT Output

```smt
; verification of impl-spec pair: some_computation ~> some_computation_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec some_computation ((hs Real)) Real
 (ite (>= hs (to_real 0)) hs (* hs (to_real -1)))
)

(declare-fun some_computation_spec (Real) Real)

; Define axioms:
(assert (forall ((hs Real)) (>= (some_computation_spec hs) (to_real 0))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((hs Real)) (=> (= hs hs) (= (some_computation hs) (some_computation_spec hs)))))
(declare-const hs Real)
(assert (= hs hs))
(assert (not (= (some_computation hs) (some_computation_spec hs))))

(check-sat)
(exit)
```

Challenges: Through writing this code, I changed the *intrinsics* module of the parser to accept negative numbers. Also how the variable bindings are treated in the backend have been resolved.

Output: *sat - the specification does not correctly define the implementation.*

Program 10

Program for testing Booleans:

Rust Program

```rust
// testing Boolean
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, SMT};

#[smt_impl]
fn _and(lhs: Boolean, rhs: Boolean) -> Boolean {
    lhs.and(rhs)
}

#[smt_spec(impls = _and)]
fn _and_spec(_lhs: Boolean, _rhs: Boolean) -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Boolean, rhs: Boolean) -> Boolean {
    if *lhs {
        _and_spec(lhs, rhs).eq(rhs)
    } else {
        _and_spec(lhs, rhs).eq(Boolean::from(false))
    }
}
```

SMT Output

```
; verification of impl-spec pair: _and ~> _and_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _and ((lhs Bool) (rhs Bool)) Bool
 (and lhs rhs)
)

(declare-fun _and_spec (Bool Bool) Bool)

; Define axioms:
(assert (forall ((lhs Bool) (rhs Bool)) (ite lhs (= (_and_spec lhs rhs) rhs) (= (_and_spec lhs rhs) false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Bool) (_rhs Bool) (lhs Bool) (rhs Bool)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_and lhs rhs) (_and_spec _lhs _rhs)))))
(declare-const _lhs Bool)
(declare-const _rhs Bool)
(declare-const lhs Bool)
(declare-const rhs Bool)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_and lhs rhs) (_and_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None; this program is relatively similar to programs 1 and 2.*

Output: *unsat - the specification correctly defines the implementation.*

Program 11

Program for testing mutually recursive functions for Integers:

Rust Program

```rust
// testing Integers mutually dependent functions
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

// these are the impl and specs
#[smt_impl(specs = _spec_add)]
pub fn _add(lhs: Integer, rhs: Integer) -> Integer {
    lhs.add(rhs)
}

#[smt_spec(impls = _add)]
pub fn _spec_add(_lhs: Integer, _rhs: Integer) -> Integer {
    unimplemented!()
}

// dummy implementations of the addition functions
#[smt_impl]
pub fn _another_add(one: Integer, two: Integer, three: Integer) -> Integer {
    one.add(two).add(three)
}

#[smt_impl]
pub fn _addtwo(one: Integer, two: Integer) -> Integer {
    _another_add(one, two, Integer::from(0))
}

#[smt_axiom]
pub fn _axiom1(lhs: Integer, rhs: Integer) -> Boolean {
    _spec_add(lhs, rhs).eq(_addtwo(lhs, rhs))
}
```

SMT Output

```
; verification of impl-spec pair: _add ~> _spec_add
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _add ((lhs Int) (rhs Int)) Int
 (+ lhs rhs)
)

(define-funs-rec ((_another_add ((one Int) (two Int) (three Int)) Int
) (_addtwo ((one Int) (two Int)) Int
)) ((+ (+ one two) three)
 (_another_add one two 0)
))

(declare-fun _spec_add (Int Int) Int)

; Define axioms:
(assert (forall ((lhs Int) (rhs Int)) (= (_spec_add lhs rhs) (_addtwo lhs rhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Int) (_rhs Int) (lhs Int) (rhs Int)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_add lhs rhs) (_spec_add _lhs _rhs)))))
(declare-const _lhs Int)
(declare-const _rhs Int)
(declare-const lhs Int)
(declare-const rhs Int)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_add lhs rhs) (_spec_add _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *This program invoked changing the whole translation to smt as the dependent functions need to be grouped together to be defined together in the smt script. Nevertheless, I have not examined what will happen when the axiom will be mutually dependent or whether that is possible at all.*

Output: *unsat - the specification correctly defines the implementation.*

Program 12

Program for testing mutually recursive functions for Integers. This is a slight mutation compared to program 11.

Rust Program

```rust
// testing Integers mutually dependent functions
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

// these are the impl and specs
#[smt_impl(specs = _spec_add)]
pub fn _add(lhs: Integer, rhs: Integer) -> Integer {
    lhs.add(rhs)
}

#[smt_spec(impls = _add)]
pub fn _spec_add(_lhs: Integer, _rhs: Integer) -> Integer {
    unimplemented!()
}

// dummy implementations of the addition functions
#[smt_impl]
pub fn _another_add(one: Integer, two: Integer, three: Integer) -> Integer {
    one.add(two).add(three)
}

#[smt_impl]
pub fn _addtwo(one: Integer, two: Integer) -> Integer {
    _another_add(one, two, Integer::from(1)) // slight mutation to prg2
}

#[smt_axiom]
pub fn _axiom1(lhs: Integer, rhs: Integer) -> Boolean {
    _spec_add(lhs, rhs).eq(_addtwo(lhs, rhs))
}
```

SMT Output

```
; verification of impl-spec pair: _add ~> _spec_add
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _add ((lhs Int) (rhs Int)) Int
 (+ lhs rhs)
)

(define-funs-rec ((_another_add ((one Int) (two Int) (three Int)) Int
) (_addtwo ((one Int) (two Int)) Int
)) ((+ (+ one two) three)
 (_another_add one two 1)
))

(declare-fun _spec_add (Int Int) Int)

; Define axioms:
(assert (forall ((lhs Int) (rhs Int)) (= (_spec_add lhs rhs) (_addtwo lhs rhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Int) (_rhs Int) (lhs Int) (rhs Int)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_add lhs rhs) (_spec_add _lhs _rhs)))))
(declare-const _lhs Int)
(declare-const _rhs Int)
(declare-const lhs Int)
(declare-const rhs Int)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_add lhs rhs) (_spec_add _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *sat - the specification does not correctly define the implementation.*

Program 13

Program for testing Real:

Rust Program

```rust
// testing Rational commutative property
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Rational, SMT};

#[smt_impl] // method cannot be defined because Rational is a system type
fn _add(lhs: Rational, rhs: Rational) -> Rational {
    lhs.add(rhs)
}

#[smt_spec(impls = _add)]
fn _add_spec(_lhs: Rational, _rhs: Rational) -> Rational {
    unimplemented!()
}

#[smt_axiom]
fn _and_axiom(lhs: Rational, rhs: Rational) -> Boolean {
    _add_spec(lhs, rhs).eq(_add(rhs, lhs))
}
```

SMT Output

```smt
; verification of impl-spec pair: _add ~> _add_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec _add ((lhs Real) (rhs Real)) Real
 (+ lhs rhs)
)

(declare-fun _add_spec (Real Real) Real)

; Define axioms:
(assert (forall ((lhs Real) (rhs Real)) (= (_add_spec lhs rhs) (_add rhs lhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Real) (_rhs Real) (lhs Real) (rhs Real)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_add lhs rhs) (_add_spec _lhs _rhs)))))
(declare-const _lhs Real)
(declare-const _rhs Real)
(declare-const lhs Real)
(declare-const rhs Real)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_add lhs rhs) (_add_spec _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 14

Program for testing mutually recursive functions for Integers.

Rust Program

```rust
// testing Integers mutually dependent functions
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_impl]
pub fn _add(lhs: Integer, rhs: Integer) -> Integer {
    lhs.add(rhs)
}

#[smt_impl]
pub fn _another_add(one: Integer, two: Integer, three: Integer) -> Integer {
    _add(_add(one, two), three)
}

#[smt_impl]
pub fn _addtwo(one: Integer, two: Integer) -> Integer {
    _another_add(one, two, Integer::from(0))
}

#[smt_spec(impls = _addtwo)]
pub fn _spec_add(_lhs: Integer, _rhs: Integer) -> Integer {
    unimplemented!()
}

#[smt_axiom]
pub fn _axiom1(lhs: Integer, rhs: Integer) -> Boolean {
    _spec_add(lhs, rhs).eq(lhs.add(rhs))
}
```

SMT Output

```
; verification of impl-spec pair: _addtwo ~> _spec_add
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-funs-rec ((_add ((lhs Int) (rhs Int)) Int
) (_another_add ((one Int) (two Int) (three Int)) Int
) (_addtwo ((one Int) (two Int)) Int
)) ((+ lhs rhs)
 (_add (_add one two) three)
 (_another_add one two 0)
))

(declare-fun _spec_add (Int Int) Int)

; Define axioms:
(assert (forall ((lhs Int) (rhs Int)) (= (_spec_add lhs rhs) (+ lhs rhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs Int) (_rhs Int) (one Int) (two Int)) (=> (and (= one _lhs) (= two _rhs)) (= (_addtwo one two) (_spec_add _lhs _rhs)))))
(declare-const _lhs Int)
(declare-const _rhs Int)
(declare-const one Int)
(declare-const two Int)
(assert (= one _lhs))
(assert (= two _rhs))
(assert (not (= (_addtwo one two) (_spec_add _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: During writing this function, the *group_dependent_funcs* function was changed as it did not group the functions correctly.

Output: *unsat - the specification correctly defines the implementation.*

Program 15

Program for testing methods and record access expressions:

Rust Program

```rust
// testing methods and record access expressions
use rusmart_smt_remark_derive::{smt_type, smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_type]
struct MyInteger {
    value: Integer,
}

#[smt_impl (method = myadd)]
fn _add(lhs: MyInteger, rhs: MyInteger) -> Integer {
    lhs.value.add(rhs.value)
}

#[smt_spec(impls = _add)]
fn _spec_add(_lhs: MyInteger, _rhs: MyInteger) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn _axiom1(lhs: MyInteger, rhs: MyInteger) -> Boolean {
    _spec_add(lhs, rhs).eq(lhs.myadd(rhs))
}
```

SMT Output

```
; verification of impl-spec pair: _add ~> _spec_add
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((MyInteger (mk-MyInteger (record_MyInteger_value_ Int)))))

; Define user functions:
(define-fun-rec _add ((lhs MyInteger) (rhs MyInteger)) Int
 (+ (record_MyInteger_value_ lhs) (record_MyInteger_value_ rhs))
)

(declare-fun _spec_add (MyInteger MyInteger) Int)

(define-fun-rec myadd ((lhs MyInteger) (rhs MyInteger)) Int
 (+ (record_MyInteger_value_ lhs) (record_MyInteger_value_ rhs))
)

; Define axioms:
(assert (forall ((lhs MyInteger) (rhs MyInteger)) (= (_spec_add lhs rhs) (myadd lhs rhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs MyInteger) (_rhs MyInteger) (lhs MyInteger) (rhs MyInteger)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_add lhs rhs) (_spec_add _lhs _rhs)))))
(declare-const _lhs MyInteger)
(declare-const _rhs MyInteger)
(declare-const lhs MyInteger)
(declare-const rhs MyInteger)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_add lhs rhs) (_spec_add _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 16

Program for testing methods, record access expressions, and generic axioms:

Rust Program

```rust
// testing struct access expression and method and generic axiom
use rusmart_smt_remark_derive::{smt_type, smt_axiom, smt_impl, smt_spec};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_type]
struct MyInteger {
    value: Integer,
}

#[smt_impl (method = myadd)]
fn _add(lhs: MyInteger, rhs: MyInteger) -> Integer {
    lhs.value.add(rhs.value)
}

#[smt_spec(impls = _add)]
fn _spec_add(_lhs: MyInteger, _rhs: MyInteger) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn _axiom1<T : SMT>(lhs: MyInteger, rhs: MyInteger) -> Boolean {
    _spec_add(lhs, rhs).eq(lhs.myadd(rhs))
}
```

SMT Output

```smt2
; verification of impl-spec pair: _add ~> _spec_add
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define Type Parameters of Function Signatures:
(declare-sort _axiom1_T 0)

; Define user-defined types:
(declare-datatypes () ((MyInteger (mk-MyInteger (record_MyInteger_value_ Int)))))

; Define user functions:
(define-fun-rec _add ((lhs MyInteger) (rhs MyInteger)) Int
 (+ (record_MyInteger_value_ lhs) (record_MyInteger_value_ rhs))
)

(declare-fun _spec_add (MyInteger MyInteger) Int)

(define-fun-rec myadd ((lhs MyInteger) (rhs MyInteger)) Int
 (+ (record_MyInteger_value_ lhs) (record_MyInteger_value_ rhs))
)

; Define axioms:
(assert (forall ((lhs MyInteger) (rhs MyInteger)) (= (_spec_add lhs rhs) (myadd lhs rhs))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_lhs MyInteger) (_rhs MyInteger) (lhs MyInteger) (rhs MyInteger)) (=> (and (= lhs _lhs) (= rhs _rhs)) (= (_add lhs rhs) (_spec_add _lhs _rhs)))))
(declare-const _lhs MyInteger)
(declare-const _rhs MyInteger)
(declare-const lhs MyInteger)
(declare-const rhs MyInteger)
(assert (= lhs _lhs))
(assert (= rhs _rhs))
(assert (not (= (_add lhs rhs) (_spec_add _lhs _rhs))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 17

Program for tuple struct access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{Boolean, Integer, SMT};

#[smt_type]
struct MyInteger(Integer, Boolean);

#[smt_impl]
fn some_impl(hs: MyInteger) -> Integer {
    if *hs.1 {
        hs.0.add(Integer::from(1))
    } else {
        hs.0
    }
}

#[smt_spec(impls = some_impl)]
fn some_spec(hs: MyInteger) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn axiom<T: SMT>(hs: MyInteger) -> Boolean {
    (some_spec(hs).eq(hs.0).and(hs.1.not()))
        .or(some_spec(hs).eq(hs.0.add(Integer::from(1))).and(hs.1))
}
```

SMT Output

```
; verification of impl-spec pair: some_impl ~> some_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define Type Parameters of Function Signatures:
(declare-sort axiom_T 0)

; Define user-defined types:
(declare-datatypes () ((MyInteger (mk-MyInteger (field_MyInteger_1_ Int) (field_MyInteger_2_ Bool)))))

; Define user functions:
(define-fun-rec some_impl ((hs MyInteger)) Int
 (ite (field_MyInteger_2_ hs) (+ (field_MyInteger_1_ hs) 1) (field_MyInteger_1_ hs))
)

(declare-fun some_spec (MyInteger) Int)

; Define axioms:
(assert (forall ((hs MyInteger)) (or (and (= (some_spec hs) (field_MyInteger_1_ hs)) (not (field_MyInteger_2_ hs))) (and (= (some_spec hs) (+ (field_MyInteger_1_ hs) 1)) (field_MyInteger_2_ hs)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((hs MyInteger)) (=> (= hs hs) (= (some_impl hs) (some_spec hs)))))
(declare-const hs MyInteger)
(assert (= hs hs))
(assert (not (= (some_impl hs) (some_spec hs))))

(check-sat)
(exit)
```

Challenges: When we write the axiom as:

```
fn axiom<T: SMT>(hs: MyInteger) -> Boolean {
    some_spec(hs).eq(hs.0) .or(some_spec(hs).eq(hs.0.add(Integer::from(1))).and(hs.1))
}
```

it will have performance issues. Also if we remove the *.and(hs.1)* from the end as well, we will have sat as there is a model satisfying the smt script (this is expected).

Output: *unsat - the specification correctly defines the implementation.*

Program 18

Enum variant unit access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
    D(Integer),
    E { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer(x1: Grades) -> Grades {
    let x = Grades::A;
    x
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(_x: Grades) -> Grades {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Grades) -> Boolean {
    grade_to_integer_spec(x).eq(Grades::A)
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)) (D (field_Grades_D_1_ Int)) (E (record_Grades_E_x_ Int) (record_Grades_E_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Grades)) Grades
 A
)

(declare-fun grade_to_integer_spec (Grades) Grades)

; Define axioms:
(assert (forall ((x Grades)) (= (grade_to_integer_spec x) A)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_x Grades) (x1 Grades)) (=> (= x1 _x) (= (grade_to_integer x1) (grade_to_integer_spec _x)))))
(declare-const _x Grades)
(declare-const x1 Grades)
(assert (= x1 _x))
(assert (not (= (grade_to_integer x1) (grade_to_integer_spec _x))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 19

Enum variant unit access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
    D(Integer),
    E { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer(x1: Grades) -> Grades {
    Grades::A
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(_x: Grades) -> Grades {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Grades) -> Boolean {
    grade_to_integer_spec(x).eq(Grades::A)
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)) (D (field_Grades_D_1_ Int)) (E (record_Grades_E_x_ Int) (record_Grades_E_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Grades)) Grades
 A
)

(declare-fun grade_to_integer_spec (Grades) Grades)

; Define axioms:
(assert (forall ((x Grades)) (= (grade_to_integer_spec x) A)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_x Grades) (x1 Grades)) (=> (= x1 _x) (= (grade_to_integer x1) (grade_to_integer_spec _x)))))
(declare-const _x Grades)
(declare-const x1 Grades)
(assert (= x1 _x))
(assert (not (= (grade_to_integer x1) (grade_to_integer_spec _x))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 20

Enum variant tuple access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
    D(Integer),
    E { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer(x1: Grades) -> Grades {
    let x = Grades::B(Integer::from(0));
    x
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(_x: Grades) -> Grades {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Grades) -> Boolean {
    grade_to_integer_spec(x).eq(Grades::B(Integer::from(0)))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)) (D (field_Grades_D_1_ Int)) (E (record_Grades_E_x_ Int) (record_Grades_E_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Grades)) Grades
 (B 0)
)

(declare-fun grade_to_integer_spec (Grades) Grades)

; Define axioms:
(assert (forall ((x Grades)) (= (grade_to_integer_spec x) (B 0))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_x Grades) (x1 Grades)) (=> (= x1 _x) (= (grade_to_integer x1) (grade_to_integer_spec _x)))))
(declare-const _x Grades)
(declare-const x1 Grades)
(assert (= x1 _x))
(assert (not (= (grade_to_integer x1) (grade_to_integer_spec _x))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 21

Enum variant tuple access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
    D(Integer),
    E { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer(x1: Grades, x2: Integer) -> Grades {
    if *x2.rem(Integer::from(2)).eq(Integer::from(0)) {
        x1 // cannot write return here
    } else {
        Grades::B(x2) // cannot write return here
    }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(x1: Grades, x2: Integer) -> Grades {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x1: Grades, x2: Integer) -> Boolean {
    (grade_to_integer_spec(x1, x2)
        .eq(x1)
        .and(x2.rem(Integer::from(2)).eq(Integer::from(0))))
    .or(grade_to_integer_spec(x1, x2)
        .eq(Grades::B(x2))
        .and(x2.rem(Integer::from(2)).eq(Integer::from(1))))
}
```

SMT Output

```smt2
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)) (D (field_Grades_D_1_ Int)) (E (record_Grades_E_x_ Int) (record_Grades_E_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Grades) (x2 Int)) Grades
 (ite (= (mod x2 2) 0) x1 (B x2))
)

(declare-fun grade_to_integer_spec (Grades Int) Grades)

; Define axioms:
(assert (forall ((x1 Grades) (x2 Int)) (or (and (= (grade_to_integer_spec x1 x2) x1) (= (mod x2 2) 0)) (and (= (grade_to_integer_spec x1 x2) (B x2)) (= (mod x2 2) 1)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x1 Grades) (x2 Int)) (=> (and (= x1 x1) (= x2 x2)) (= (grade_to_integer x1 x2) (grade_to_integer_spec x1 x2)))))
(declare-const x1 Grades)
(declare-const x2 Int)
(assert (= x1 x1))
(assert (= x2 x2))
(assert (not (= (grade_to_integer x1 x2) (grade_to_integer_spec x1 x2))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 22

Enum variant record access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
    D(Integer),
    E { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer() -> Grades {
    let x = Integer::from(0);
    let y = Integer::from(1);
    Grades::C { x, y }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec() -> Grades {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x1: Integer, x2: Integer) -> Boolean {
    x1.eq(Integer::from(0))
        .and(x2.eq(Integer::from(1)))
        .implies(grade_to_integer_spec().eq(Grades::C { x: x1, y: x2 }))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)) (D (field_Grades_D_1_ Int)) (E (record_Grades_E_x_ Int) (record_Grades_E_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer () Grades
 (C 0 1)
)

(declare-fun grade_to_integer_spec () Grades)

; Define axioms:
(assert (forall ((x1 Int) (x2 Int)) (=> (and (= x1 0) (= x2 1)) (= grade_to_integer_spec (C x1 x2)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (grade_to_integer ) (grade_to_integer_spec )))))
(assert (not (= grade_to_integer grade_to_integer_spec)))

(check-sat)
(exit)
```

Challenges: added *DataType::Enum* to:

```rust
fn expect_type_record(&self, sort_id: UsrSortId) -> BTreeMap<String, Sort> {
        match self.parent.ir.ty_registry.retrieve(sort_id) {
            DataType::Record(record) => record.clone(),
            DataType::Enum(adt) => {
                let mut record = BTreeMap::new();
                for (_, variant) in adt.iter() {
                    if let Variant::Record(r) = variant {
                        record.extend(r.clone());
                    }
                }
                record
            }
            dt => panic!("type mismatch: expect <record> | actual {}", dt),
```

```
        }
    }
```

otherwise, *test panicked: type mismatch: expect ¡record¿ ZZZZ— actual [A(),B(Integer),Cx:Integer,y:Integer,D(Integer* error was invoked from *exp.rs* in ir. Accordingly updated the tuple one, but no error was found when we give the enum tuple(?)

Output: *unsat - the specification correctly defines the implementation.*

Program 23

Enum variant tuple access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
    D(Integer),
    E { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer() -> Grades {
    let x = Integer::from(0);
    Grades::D(x)
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec() -> Grades {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Integer) -> Boolean {
    x.eq(Integer::from(0))
        .implies(grade_to_integer_spec().eq(Grades::D(x)))
}
```

SMT Output

```smt
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)) (D (field_Grades_D_1_ Int)) (E (record_Grades_E_x_ Int) (record_Grades_E_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer () Grades
 (D 0)
)

(declare-fun grade_to_integer_spec () Grades)

; Define axioms:
(assert (forall ((x Int)) (=> (= x 0) (= grade_to_integer_spec (D x)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=> (= (grade_to_integer ) (grade_to_integer_spec )))))
(assert (not (= grade_to_integer grade_to_integer_spec)))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 24

Struct tuple definition & access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
struct MyIntBool(Integer, Boolean);

#[smt_impl]
fn grade_to_integer(x: Integer) -> Integer {
    // we cannot destructure the tuple here (error: unrecognized pattern for declaration - expect an identifier or a tuple)
    // let MyIntBool(o,p) = MyIntBool(x, Boolean::from(true));
    let var = MyIntBool(x, Boolean::from(true));
    var.0
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(x: Integer) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Integer) -> Boolean {
    grade_to_integer_spec(x).le(Integer::from(3))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((MyIntBool (mk-MyIntBool (field_MyIntBool_1_ Int) (field_MyIntBool_2_ Bool)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x Int)) Int
 (field_MyIntBool_1_ (mk-MyIntBool x true))
)

(declare-fun grade_to_integer_spec (Int) Int)

; Define axioms:
(assert (forall ((x Int)) (<= (grade_to_integer_spec x) 3)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x Int)) (=> (= x x) (= (grade_to_integer x) (grade_to_integer_spec x)))))
(declare-const x Int)
(assert (= x x))
(assert (not (= (grade_to_integer x) (grade_to_integer_spec x))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *sat - the specification does not correctly define the implementation.*

Program 25

Struct record definition & access:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
struct MyIntBool {
    a: Integer,
    b: Boolean,
}

#[smt_impl]
fn grade_to_integer(x: Integer) -> Integer {
    // we cannot destructure the tuple here (error: unrecognized pattern for declaration - expect an identifier or a tuple)
    // let MyIntBool{
    //     a,
    //     b,
    // } = MyIntBool{
    //     a: x,
    //     b: Boolean::from(true),
    // };
    let var = MyIntBool {
        a: x,
        b: Boolean::from(true),
    };
    var.a
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(x: Integer) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Integer) -> Boolean {
    grade_to_integer_spec(x).le(Integer::from(3))
}
```

SMT Output

```smt
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((MyIntBool (mk-MyIntBool (record_MyIntBool_a_ Int) (record_MyIntBool_b_ Bool)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x Int)) Int
 (record_MyIntBool_a_ (mk-MyIntBool x true))
)

(declare-fun grade_to_integer_spec (Int) Int)

; Define axioms:
(assert (forall ((x Int)) (<= (grade_to_integer_spec x) 3)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x Int)) (=> (= x x) (= (grade_to_integer x) (grade_to_integer_spec x)))))
(declare-const x Int)
(assert (= x x))
(assert (not (= (grade_to_integer x) (grade_to_integer_spec x))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *sat - the specification does not correctly define the implementation.*

Program 26

Enum definition:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum IntBool {
    Int(Integer),
    Bool(Boolean),
}

#[smt_impl]
fn grade_to_integer(x1: Integer, x2: Boolean) -> IntBool {
    let var1 = IntBool::Int(x1);
    let var2 = IntBool::Bool(x2);
    if *x2 {
        var1
    } else {
        var2
    }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(x1: Integer, x2: Boolean) -> IntBool {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x1: Integer, x2: Boolean) -> Boolean {
    (grade_to_integer_spec(x1, x2).eq(IntBool::Int(x1)).and(x2)).or(grade_to_integer_spec(x1, x2)
        .eq(IntBool::Bool(x2))
        .and(x2.not()))
}
```

SMT Output

```smt
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((IntBool (Bool (field_IntBool_Bool_1_ Bool)) (Int (field_IntBool_Int_1_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Int) (x2 Bool)) IntBool
 (ite x2 (Int x1) (Bool x2))
)

(declare-fun grade_to_integer_spec (Int Bool) IntBool)

; Define axioms:
(assert (forall ((x1 Int) (x2 Bool)) (or (and (= (grade_to_integer_spec x1 x2) (Int x1)) x2) (and (= (grade_to_integer_spec x1 x2) (Bool x2)) (not x2)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x1 Int) (x2 Bool)) (=> (and (= x1 x1) (= x2 x2)) (= (grade_to_integer x1 x2) (grade_to_integer_spec x1 x2)))))
(declare-const x1 Int)
(declare-const x2 Bool)
(assert (= x1 x1))
(assert (= x2 x2))
(assert (not (= (grade_to_integer x1 x2) (grade_to_integer_spec x1 x2))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 27

Enum definition:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum IntBool {
    Unit,
    Int { a: Integer },
    Bool(Boolean),
}

#[smt_impl]
fn grade_to_integer(x1: Integer, x2: Boolean) -> IntBool {
    let var1 = IntBool::Int { a: x1 };
    let var2 = IntBool::Bool(x2);
    let var3 = IntBool::Unit;
    if *x2 {
        if *x1.ge(Integer::from(0)) {
            var1
        } else {
            var3
        }
    } else {
        var2
    }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(x1: Integer, x2: Boolean) -> IntBool {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x1: Integer, x2: Boolean) -> Boolean {
    let z = IntBool::Unit;
    (grade_to_integer_spec(x1, x2)
        .eq(IntBool::Int { a: x1 })
        .and(x2))
    .or(grade_to_integer_spec(x1, x2)
        .eq(IntBool::Bool(x2))
        .and(x2.not()))
    .or(grade_to_integer_spec(x1, x2)
        .eq(z)
        .and(x2)
        .and(x1.lt(Integer::from(0))))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((IntBool (Bool (field_IntBool_Bool_1_ Bool)) (Int (record_IntBool_Int_a_ Int)) (Unit))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Int) (x2 Bool)) IntBool
 (ite x2 (ite (>= x1 0) (Int x1) Unit) (Bool x2))
)

(declare-fun grade_to_integer_spec (Int Bool) IntBool)

; Define axioms:
(assert (forall ((x1 Int) (x2 Bool)) (or (or (and (= (grade_to_integer_spec x1 x2) (Int x1)) x2) (and (= (grade_to_integer_spec x1 x2) (Bool x2)) (not x2))) (and (and (= (grade_to_integer_spec x1 x2) Unit) x2) (< x1 0)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x1 Int) (x2 Bool)) (=> (and (= x1 x1) (= x2 x2)) (= (grade_to_integer x1 x2) (grade_to_integer_spec x1 x2)))))
(declare-const x1 Int)
(declare-const x2 Bool)
(assert (= x1 x1))
(assert (= x2 x2))
(assert (not (= (grade_to_integer x1 x2) (grade_to_integer_spec x1 x2))))

(check-sat)
(exit)
```

Challenges: Performance issues; no result was given - expected unsat.

Output: *unknown - solver z3 cannot define whether the specification matches the implementation.*

Program 28

Match expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer(x1: Grades) -> Integer {
    match x1 {
        Grades::A => Integer::from(1),
        Grades::B(i) => i,
        Grades::C { x, y } => x.add(y),
    }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(_x: Grades) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Grades) -> Boolean {
    (x.eq(Grades::A).implies(grade_to_integer_spec(x).eq(Integer::from(1))))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Grades)) Int
 (ite (is-A x1) 1 (ite (is-B x1) (field_Grades_B_1_ x1) (+ (record_Grades_C_x_ x1) (record_Grades_C_y_ x1))))
)

(declare-fun grade_to_integer_spec (Grades) Int)

; Define axioms:
(assert (forall ((x Grades)) (=> (= x A) (= (grade_to_integer_spec x) 1))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_x Grades) (x1 Grades)) (=> (= x1 _x) (= (grade_to_integer x1) (grade_to_integer_spec _x)))))
(declare-const _x Grades)
(declare-const x1 Grades)
(assert (= x1 _x))
(assert (not (= (grade_to_integer x1) (grade_to_integer_spec _x))))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *sat - the specification does not correctly define the implementation.*

Program 29

Match expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_type]
enum Grades {
    A,
    B(Integer),
    C { x: Integer, y: Integer },
}

#[smt_impl]
fn grade_to_integer(x1: Grades) -> Integer {
    match x1 {
        Grades::A => Integer::from(1),
        Grades::B(i) => i,
        Grades::C { x, y } => x.add(y),
    }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(_x: Grades) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Grades) -> Boolean {
    match x {
        Grades::A => grade_to_integer_spec(x).eq(Integer::from(0)),
        Grades::B(i) => grade_to_integer_spec(x).eq(i),
        Grades::C { x: x1, y: y1 } => grade_to_integer_spec(x).eq(x1.add(y1)),
    }
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Grades (A) (B (field_Grades_B_1_ Int)) (C (record_Grades_C_x_ Int) (record_Grades_C_y_ Int)))))

; Define user functions:
(define-fun-rec grade_to_integer ((x1 Grades)) Int
 (ite (is-A x1) 1 (ite (is-B x1) (field_Grades_B_1_ x1) (+ (record_Grades_C_x_ x1) (record_Grades_C_y_ x1))))
)

(declare-fun grade_to_integer_spec (Grades) Int)

; Define axioms:
(assert (forall ((x Grades)) (ite (is-A x) (= (grade_to_integer_spec x) 0) (ite (is-B x) (= (grade_to_integer_spec x) (field_Grades_B_1_ x)) (= (grade_to_integer_spec x) (+ (record_Grades_C_x_ x) (record_Grades_C_y_ x)))))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_x Grades) (x1 Grades)) (=> (= x1 _x) (= (grade_to_integer x1) (grade_to_integer_spec _x)))))
(declare-const _x Grades)
(declare-const x1 Grades)
(assert (= x1 _x))
(assert (not (= (grade_to_integer x1) (grade_to_integer_spec _x))))

(check-sat)
(exit)
```

Challenges: *Performance issues*; expected unsat got nothing.

Output: *unknown - solver z3 cannot define whether the specification matches the implementation.*

Program 30

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Text, SMT};

#[smt_impl]
fn grade_to_integer(x1: Text) -> Integer {
    let A: Text = Text::from("A");
    let B: Text = Text::from("B");
    let C: Text = Text::from("C");

    // do not write as this because the else if will be ignored by the parser
    // if *x1.eq(A) {
    //     Integer::from(0)
    // } else if *x1.eq(B) {
    //     Integer::from(1)
    // } else if *x1.eq(C) {
    //     Integer::from(2)
    // } else {
    //     Integer::from(-1)
    // }
    if *x1.eq(A) {
        Integer::from(0)
    } else {
        if *x1.eq(B) {
            Integer::from(1)
        } else {
            if *x1.eq(C) {
                Integer::from(2)
            } else {
                Integer::from(-1)
            }
        }
    }
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec(_x: Text) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom(x: Text) -> Boolean {
    x.eq(Text::from("A"))
        .implies(grade_to_integer_spec(x).eq(Integer::from(0)))
        .and(x
            .eq(Text::from("B"))
            .implies(grade_to_integer_spec(x).eq(Integer::from(1))))
        .and(x
            .eq(Text::from("C"))
            .implies(grade_to_integer_spec(x).eq(Integer::from(2))))
        .and((x.gt(Text::from("C")).or(x.lt(Text::from("A"))))
            .implies(grade_to_integer_spec(x).eq(Integer::from(-1))))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec grade_to_integer ((x1 String)) Int
 (ite (= x1 "A") 0 (ite (= x1 "B") 1 (ite (= x1 "C") 2 -1)))
)

(declare-fun grade_to_integer_spec (String) Int)

; Define axioms:
(assert (forall ((x String)) (and (and (and (=> (= x "A") (= (grade_to_integer_spec x) 0)) (=> (= x "B") (= (grade_to_integer_spec x) 1))) (=> (= x "C") (= (grade_to_integer_spec x) 2))) (=> (or (str.< "C" x) (str.< x "A")) (= (grade_to_integer_spec x) -1)))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((_x String) (x1 String)) (=> (= x1 _x) (= (grade_to_integer x1) (grade_to_integer_spec _x)))))
(declare-const _x String)
(declare-const x1 String)
(assert (= x1 _x))
(assert (not (= (grade_to_integer x1) (grade_to_integer_spec _x))))

(check-sat)
(exit)
```

Challenges: *Performance issues for string expressions*; expected unsat got nothing. Whereas if we wrote

```
(x.ne(Text::from("C")).and(x.ne(Text::from("A"))).and(x.ne(Text::from("B")))
```

in line 50 we would have got unsat.

Output: *unknown - solver z3 cannot define whether the specification matches the implementation.*

Program 31

Cloak expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Text, SMT};

#[smt_impl]
fn grade_to_integer() -> Integer {
    // cannot write let a = Cloak::shield(Integer::from(0)); as we will get incomplete type error
    let a: Cloak<Integer> = Cloak::shield(Integer::from(0));
    let b = a.reveal();
    b
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom() -> Boolean {
    grade_to_integer_spec().eq(Integer::from(0))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(declare-sort Cloak 1) ; Cloak<T>
(declare-fun shield (Int) (Cloak Int))
(declare-fun reveal ((Cloak Int)) Int)
(assert (forall ((x (Cloak Int))) (= (shield (reveal x)) x))) ; shield(reveal(x)) = x
(assert (forall ((x Int)) (= (reveal (shield x)) x))) ; reveal(shield(x)) = x
(define-fun-rec grade_to_integer () Int
 (reveal (shield 0))
)

(declare-fun grade_to_integer_spec () Int)

; Define axioms:
(assert (= grade_to_integer_spec 0))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (grade_to_integer ) (grade_to_integer_spec )))))
(assert (not (= grade_to_integer grade_to_integer_spec)))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 32

Cloak expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Text, SMT};

#[smt_impl]
fn grade_to_integer() -> Boolean {
    // cannot write let a = Cloak::shield(Integer::from(0)); as we will get incomplete type error
    let a: Cloak<Integer> = Cloak::shield(Integer::from(0));
    let b = a.reveal();
    let c: Cloak<Integer> = Cloak::shield(b);
    let d = c.reveal();
    b.eq(d)
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec() -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom() -> Boolean {
    grade_to_integer_spec().eq(Boolean::from(true))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(declare-sort Cloak 1) ; Cloak<T>
(declare-fun shield (Int) (Cloak Int))
(declare-fun reveal ((Cloak Int)) Int)
(assert (forall ((x (Cloak Int))) (= (shield (reveal x)) x))) ; shield(reveal(x)) = x
(assert (forall ((x Int)) (= (reveal (shield x)) x))) ; reveal(shield(x)) = x
(define-fun-rec grade_to_integer () Bool
 (= (reveal (shield 0)) (reveal (shield (reveal (shield 0))))))
)

(declare-fun grade_to_integer_spec () Bool)

; Define axioms:
(assert (= grade_to_integer_spec true))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (grade_to_integer ) (grade_to_integer_spec )))))
(assert (not (= grade_to_integer grade_to_integer_spec)))

(check-sat)
(exit)
```

Challenges: For now the limitation is that all compound rusmart types need to be annotated. If not in this case, we will get *test panicked: Non-disjoint equivalence sets* coming from infer.rs of the parser.

Output: *unsat - the specification correctly defines the implementation.*

Program 33

Seq expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Seq, Text, SMT};

#[smt_impl]
fn grade_to_integer() -> Integer {
    // cannot write let a = Seq::new(); as we will get incomplete type error
    let a: Seq<Integer> = Seq::new();
    // must have type annotation
    let b: Seq<Integer> = a.append(Integer::from(0));
    b.length()
}

#[smt_spec(impls = grade_to_integer)]
fn grade_to_integer_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom() -> Boolean {
    grade_to_integer_spec().eq(Integer::from(1))
}
```

SMT Output

```
; verification of impl-spec pair: grade_to_integer ~> grade_to_integer_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(declare-const seq_0 (Seq Int))
(assert (= seq_0 (as seq.empty (Seq Int)))) ; seq.empty
(define-fun-rec grade_to_integer () Int
 (seq.len (seq.++ seq_0 (seq.unit 0)))
)

(declare-fun grade_to_integer_spec () Int)

; Define axioms:
(assert (= grade_to_integer_spec 1))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (grade_to_integer ) (grade_to_integer_spec )))))
(assert (not (= grade_to_integer grade_to_integer_spec)))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 34

Seq expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Seq, Text, SMT};

#[smt_impl]
fn len_elem() -> (Integer, Integer) {
    // cannot write let a = Seq::new(); as we will get incomplete type error
    let a: Seq<Integer> = Seq::new();
    // must have type annotation
    let b: Seq<Integer> = a.append(Integer::from(0));
    let c = b.at_unchecked(0.into());
    (b.length(), c)
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> (Integer, Integer) {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom() -> Boolean {
    len_elem_spec().eq((Integer::from(1), Integer::from(0)))
}
```

SMT Output

```smt
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes ()
 ((Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer
 (field_Tuple_Integer_Integer_1_ Int)
 (field_Tuple_Integer_Integer_2_ Int)))))

; Define user functions:
(declare-const seq_0 (Seq Int))
(assert (= seq_0 (as seq.empty (Seq Int)))) ; seq.empty
(define-fun-rec len_elem ()
 Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer (seq.len (seq.++
  seq_0 (seq.unit 0))) (seq.nth (seq.++
  seq_0 (seq.unit 0)) 0))
)

(declare-fun len_elem_spec ()
 Tuple_Integer_Integer)

; Define axioms:
(assert (= len_elem_spec
 (mk-Tuple_Integer_Integer 1 0)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 35

Seq expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Seq, Text, SMT};

#[smt_impl]
fn len_elem() -> (Integer, Integer) {
    // cannot write let a = Seq::new(); as we will get incomplete type error
    let a: Seq<Integer> = Seq::new();
    // must have type annotation
    let b: Seq<Integer> = a.append(Integer::from(0));
    let c = a.at_unchecked(0.into());
    (b.length(), c)
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> (Integer, Integer) {
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom() -> Boolean {
    len_elem_spec().eq((Integer::from(1), Integer::from(0)))
}
```

SMT Output

```
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes ()
 ((Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer
 (field_Tuple_Integer_Integer_1_ Int)
 (field_Tuple_Integer_Integer_2_ Int)))))

; Define user functions:
(declare-const seq_0 (Seq Int))
(assert (= seq_0 (as seq.empty (Seq Int)))) ; seq.empty
(define-fun-rec len_elem ()
 Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer (seq.len (seq.++ seq_0 (seq.unit 0))) (seq.nth seq_0 0))
)

(declare-fun len_elem_spec ()
 Tuple_Integer_Integer)

; Define axioms:
(assert (= len_elem_spec
 (mk-Tuple_Integer_Integer 1 0)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *sat - the specification does not correctly define the implementation.*

Program 36

Seq expressions:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Seq, Text, SMT};

#[smt_impl]
fn len_elem() -> (Integer, Integer, Boolean) {
    // cannot write let a = Seq::new(); as we will get incomplete type error
    let a: Seq<Integer> = Seq::new();
    // must have type annotation
    let b: Seq<Integer> = a.append(Integer::from(0));
    let c = b.at_unchecked(0.into());
    let d = b.includes(c);
    (b.length(), c, d)
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> (Integer, Integer, Boolean){
    unimplemented!()
}

#[smt_axiom]
fn grade_to_integer_axiom() -> Boolean {
    len_elem_spec().eq((Integer::from(1), Integer::from(0), Boolean::from(true)))
}
```

SMT Output

```smt
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes ()
 ((Tuple_Integer_Integer_Boolean
 (mk-Tuple_Integer_Integer_Boolean
 (field_Tuple_Integer_Integer_Boolean_1_Int)
 (field_Tuple_Integer_Integer_Boolean_2_Int)
 (field_Tuple_Integer_Integer_Boolean_3_Bool)))))

; Define user functions:
(declare-const seq_0 (Seq Int))
(assert (= seq_0 (as seq.empty (Seq Int)))) ; seq.empty
(define-fun-rec len_elem ()
 Tuple_Integer_Integer_Boolean
 (mk-Tuple_Integer_Integer_Boolean (seq.len (seq.++ seq_0 (seq.unit 0))) (seq.nth (seq.++ seq_0 (seq.unit 0)) 0) (seq.contains (seq.++ seq_0 (seq.unit 0)) (seq.unit (seq.nth (seq.++ seq_0 (seq.unit 0)) 0))))
)

(declare-fun len_elem_spec ()
 Tuple_Integer_Integer_Boolean)

; Define axioms:
(assert (= len_elem_spec
 (mk-Tuple_Integer_Integer_Boolean 1 0 true)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: *None.*

Output: *unsat - the specification correctly defines the implementation.*

Program 37

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Set, Text, SMT};

#[smt_impl]
fn len_elem() -> Set<Integer> {
    // cannot write let a = Seq::new(); as we will get incomplete type error
    let a: Set<Integer> = Set::new();
    // must have type annotation
    let b: Set<Integer> = a.insert(Integer::from(0));
    b
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> Set<Integer> {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    len_elem_spec().contains(Integer::from(0 ))
}
```

SMT Output

```smt
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(declare-const set_0 (Set Int))
(assert (forall ((x Int)) (= (select set_0 x) false))) ; set.empty
(declare-fun len ((Set Int)) Int)
(assert (= (len set_0) 0)) ; length of empty set is 0
(assert (forall ((m (Set Int)) (i Int))
                                (=> (not (select m i)) (= (len (store m i true)) (+ (len m) 1))))) ; length of set after adding an element
(assert (forall ((m (Set Int)) (i Int))
                                (=> (select m i) (= (len (store m i true)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                                (=> (select m i) (= (len (store m i false)) (- (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                                (=> (not (select m i)) (= (len (store m i false)) (len m)))))
(define-fun-rec len_elem () (Set Int)
 (store set_0 0 true)
)

(declare-fun len_elem_spec () (Set Int))

; Define axioms:
(assert (select len_elem_spec 0))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: *Performance issues; expecting unsat but no result given.* Even if we make the axiom tighter: `len_elem_spec().contains(Integer::from(0)).and(len_elem_spec().length().eq(Integer::from(1)))` still no output will be given.

Output: *unknown - solver z3 cannot define whether the specification matches the implementation.*

Program 38

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Integer, Set, Text, SMT};

#[smt_impl]
fn len_elem() -> Boolean {
    // cannot write let a = Seq::new(); as we will get incomplete type error
    let a: Set<Integer> = Set::new();
    // must have type annotation
    let b: Set<Integer> = a.insert(Integer::from(0));
    b.contains(Integer::from(0))
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    len_elem_spec()
}
```

SMT Output

```
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(declare-const set_0 (Set Int))
(assert (forall ((x Int)) (= (select set_0 x) false))) ; set.empty
(declare-fun len ((Set Int)) Int)
(assert (= (len set_0) 0)) ; length of empty set is 0
(assert (forall ((m (Set Int)) (i Int))
                                (=> (not (select m i)) (= (len (store m i true)) (+ (len m) 1))))) ; length of set after adding an element
(assert (forall ((m (Set Int)) (i Int))
                                (=> (select m i) (= (len (store m i true)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                                (=> (select m i) (= (len (store m i false)) (- (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                                (=> (not (select m i)) (= (len (store m i false)) (len m)))))
(define-fun-rec len_elem () Bool
 (select (store set_0 0 true) 0)
)

(declare-fun len_elem_spec () Bool)

; Define axioms:
(assert len_elem_spec)

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: Same program as 37, but without issues.

Output: *unsat - the specification correctly defines the implementation.*

Program 39

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Error, Integer, Map, Set, Text, SMT};

#[smt_impl]
fn len_elem() -> (Integer, Integer) {
    let m: Map<Integer, Set<Integer>> = Map::new();
    let s: Set<Integer> = Set::new();
    let i = Integer::from(0);

    let new_m: Map<Integer, Set<Integer>> = m.put_unchecked(i, s);
    let r1 = new_m.get_unchecked(i).length();
    let new_s: Set<Integer> = s.insert(i);
    let new_m2: Map<Integer, Set<Integer>> = m.put_unchecked(i, new_s);
    let r2 = new_m2.get_unchecked(i).length();

    (r1, r2)
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> (Integer, Integer) {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    // len_elem_spec()
    //     .0
    //     .eq(Integer::from(0))
    //     .and(len_elem_spec().1.eq(Integer::from(1)))
    len_elem_spec().eq((Integer::from(0), Integer::from(1)))
}
```

SMT Output

```
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Tuple_Integer_Integer (mk-Tuple_Integer_Integer
↪  (field_Tuple_Integer_Integer_1_ Int) (field_Tuple_Integer_Integer_2_ Int)))))

; Define user functions:
(declare-const map_0 (Array Int (Set Int)))
(declare-const not_present_Set_Int (Set Int))
(assert (forall ((x Int)) (= (select map_0 x) not_present_Set_Int))) ; array.empty
(declare-fun len_map ((Array Int (Set Int))) Int)
(assert (= (len_map map_0) 0)) ; length of empty map is 0
(define-fun in_map ((m (Array Int (Set Int))) (i Int)) Bool
                            (not (= (select m i) not_present_Set_Int)))
(assert (forall ((m (Array Int (Set Int))) (i Int) (v (Set Int)))
                        (=> (and (not (in_map m i)) (not (= v not_present_Set_Int))) (=
                        ↪  (len_map (store m i v)) (+ (len_map m) 1)))))
(assert (forall ((m (Array Int (Set Int))) (i Int) (v (Set Int)))
```

```
                              (=> (and (in_map m i) (not (= v not_present_Set_Int))) (=
                              ↪  (len_map (store m i v)) (len_map m)))))
(assert (forall ((m (Array Int (Set Int))) (i Int))
                                    (=> (in_map m i)
                                        (= (len_map (store m i not_present_Set_Int)) (-
                                        ↪  (len_map m) 1)))))
(assert (forall ((m (Array Int (Set Int))) (i Int))
                                    (=> (not (in_map m i))
                                        (= (len_map (store m i not_present_Set_Int)) (len_map
                                        ↪  m)))))
(declare-const set_1 (Set Int))
(assert (forall ((x Int)) (= (select set_1 x) false))) ; set.empty
(declare-fun len ((Set Int)) Int)
(assert (= (len set_1) 0)) ; length of empty set is 0
(assert (forall ((m (Set Int)) (i Int))
                              (=> (not (select m i)) (= (len (store m i true)) (+ (len m)
                              ↪  1))))) ; length of set after adding an element
(assert (forall ((m (Set Int)) (i Int))
                              (=> (select m i) (= (len (store m i true)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                              (=> (select m i) (= (len (store m i false)) (- (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                              (=> (not (select m i)) (= (len (store m i false)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (not (select m i)) (= (len (store m i true)) (+ (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (select m i) (= (len (store m i true)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (select m i) (= (len (store m i false)) (- (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (not (select m i)) (= (len (store m i false)) (len m)))))
(define-fun-rec len_elem () Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer (len (select (store map_0 0 set_1) 0)) (len (select (store map_0 0
 ↪  (store set_1 0 true)) 0)))
)

(declare-fun len_elem_spec () Tuple_Integer_Integer)

; Define axioms:
(assert (= len_elem_spec (mk-Tuple_Integer_Integer 0 1)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: No issues, but if we wrote

```
len_elem_spec()
  .0
  .eq(Integer::from(0))
  .and(len_elem_spec().1.eq(Integer::from(1)))
```

instead of

```
len_elem_spec().eq((Integer::from(0), Integer::from(1)))
```

we will get an error. This could be fixed by changing the expr.rs

```
TypeRef::Pack(s) => &TypeBody::Tuple(TypeTuple {
    slots: s
        .iter()
        .map(|t| unifier.refresh_type(t).reverse().expect("expected a type"))
        .collect::<Vec<TypeTag>>(),
}),
```

But we have kept the parser such that tuple variables cannot be accessed.

Output: *unsat - the specification correctly defines the implementation.*

Program 40

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Error, Integer, Map, Set, Text, SMT};

#[smt_impl]
fn len_elem() -> (
    Integer,
    Integer,
    Integer,
    Boolean,
    Boolean,
    Boolean,
    Boolean,
) {
    let m: Map<Integer, Set<Integer>> = Map::new();
    let s: Set<Integer> = Set::new();
    let i = Integer::from(0);

    let new_m: Map<Integer, Set<Integer>> = m.put_unchecked(i, s);
    let r1 = new_m.get_unchecked(i).length();
    let new_s: Set<Integer> = s.insert(i);
    let new_m2: Map<Integer, Set<Integer>> = m.put_unchecked(i, new_s);
    let r2 = new_m2.get_unchecked(i).length();
    let new_m3: Map<Integer, Set<Integer>> = m.del_unchecked(i);
    let r3 = new_m3.get_unchecked(i).length();

    let b1 = m.contains_key(i);
    let b2 = new_m.contains_key(i);
    let b3 = new_m2.contains_key(i);
    let b4 = new_m3.contains_key(i);
    (r1, r2, r3, b1, b2, b3, b4)
}

#[smt_spec(impls = len_elem)]
fn len_elem_spec() -> (
    Integer,
    Integer,
    Integer,
    Boolean,
    Boolean,
    Boolean,
    Boolean,
) {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    len_elem_spec().eq((
        Integer::from(0),
        Integer::from(1),
        Integer::from(0),
        Boolean::from(false),
        Boolean::from(true),
        Boolean::from(true),
        Boolean::from(false),
    ))
```

```
}
```

SMT Output

```
; verification of impl-spec pair: len_elem ~> len_elem_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean
  (mk-Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_1_ Int)
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_2_ Int)
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_3_ Int)
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_4_ Bool)
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_5_ Bool)
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_6_ Bool)
  (field_Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean_7_ Bool)))))

; Define user functions:
(declare-const map_0 (Array Int (Set Int)))
(declare-const not_present_Set_Int (Set Int))
(assert (forall ((x Int)) (= (select map_0 x) not_present_Set_Int))) ; array.empty
(declare-fun len_map ((Array Int (Set Int))) Int)
(assert (= (len_map map_0) 0)) ; length of empty map is 0
(define-fun in_map ((m (Array Int (Set Int))) (i Int)) Bool
                            (not (= (select m i) not_present_Set_Int)))
(assert (forall ((m (Array Int (Set Int))) (i Int) (v (Set Int)))
                        (=> (and (not (in_map m i)) (not (= v not_present_Set_Int))) (=
                        ↪  (len_map (store m i v)) (+ (len_map m) 1)))))
(assert (forall ((m (Array Int (Set Int))) (i Int) (v (Set Int)))
                            (=> (and (in_map m i) (not (= v not_present_Set_Int))) (=
                            ↪  (len_map (store m i v)) (len_map m)))))
(assert (forall ((m (Array Int (Set Int))) (i Int))
                                    (=> (in_map m i)
                                        (= (len_map (store m i not_present_Set_Int)) (-
                                        ↪  (len_map m) 1)))))
(assert (forall ((m (Array Int (Set Int))) (i Int))
                                    (=> (not (in_map m i))
                                        (= (len_map (store m i not_present_Set_Int)) (len_map
                                        ↪  m)))))
(declare-const set_1 (Set Int))
(assert (forall ((x Int)) (= (select set_1 x) false))) ; set.empty
(declare-fun len ((Set Int)) Int)
(assert (= (len set_1) 0)) ; length of empty set is 0
(assert (forall ((m (Set Int)) (i Int))
                            (=> (not (select m i)) (= (len (store m i true)) (+ (len m)
                            ↪  1))))) ; length of set after adding an element
(assert (forall ((m (Set Int)) (i Int))
                            (=> (select m i) (= (len (store m i true)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                            (=> (select m i) (= (len (store m i false)) (- (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                            (=> (not (select m i)) (= (len (store m i false)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (not (select m i)) (= (len (store m i true)) (+ (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
```

```
                (=> (select m i) (= (len (store m i true)) (len m)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (select m i) (= (len (store m i false)) (- (len m) 1)))))
(assert (forall ((m (Set Int)) (i Int))
                (=> (not (select m i)) (= (len (store m i false)) (len m)))))
(define-fun-rec len_elem () Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean
 (mk-Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean (len (select (store map_0
 ↪  0 set_1) 0)) (len (select (store map_0 0 (store set_1 0 true)) 0)) (len (select (store
 ↪  map_0 0 not_present_Set_Int) 0)) (distinct (select map_0 0) not_present_Set_Int)
 ↪  (distinct (select (store map_0 0 set_1) 0) not_present_Set_Int) (distinct (select (store
 ↪  map_0 0 (store set_1 0 true)) 0) not_present_Set_Int) (distinct (select (store map_0 0
 ↪  not_present_Set_Int) 0) not_present_Set_Int))
)

(declare-fun len_elem_spec () Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean)

; Define axioms:
(assert (= len_elem_spec (mk-Tuple_Integer_Integer_Integer_Boolean_Boolean_Boolean_Boolean 0
↪  1 0 false true true false)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (len_elem ) (len_elem_spec )))))
(assert (not (= len_elem len_elem_spec)))

(check-sat)
(exit)
```

Challenges: *Performance issue; expected unsat.*

Output: *unknown - solver z3 cannot define whether the specification matches the implementation.*

Program 41

Accessing tuple with unpacking:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Error, Integer, Map, Set, Text, SMT};

#[smt_impl]
fn pack() -> Integer {
    let y= (Integer::from(1), Integer::from(1));
    let x = Integer::from(0);
    Integer::from(0)
}

#[smt_spec(impls = pack)]
fn pack_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    pack_spec().eq(Integer::from(0))
}
```

SMT Output

```
; verification of impl-spec pair: pack ~> pack_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes ()
 ((Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer
 (field_Tuple_Integer_Integer_1_ Int)
 (field_Tuple_Integer_Integer_2_ Int)))))

; Define user functions:
(define-fun-rec pack () Int
 0
)

(declare-fun pack_spec () Int)

; Define axioms:
(assert (= pack_spec 0))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (pack ) (pack_spec )))))
(assert (not (= pack pack_spec)))

(check-sat)
(exit)
```

Challenges:

```
TypeRef::Pack(s) => &TypeBody::Tuple(TypeTuple {
    slots: s
        .iter()
        .map(|t| unifier.refresh_type(t).reverse().expect("expected a type"))
        .collect::<Vec<TypeTag>>(),
}),
```

If this was added to the *expr.rs* module of the parser, then instead of *Integer::from(0)* we could have written *y.0*. However, the issue is much bigger than that!

```
let y: (Integer, Integer)= (Integer::from(1), Integer::from(1));
```

Should not be possible and if we give it an explicit type annotation the parser crashes! But if we do not it will pass. Again, this is because the infer.rs module has bugs. Also we can access a tuple struct but not a tuple is that okay and desired? Lastly to allow *y: (Integer, Integer)* we can comment out:

```
if matches!(t, TypeTag::Pack(_)) {
    bail_on!(ident, "expect a non-pack type");
}
```

from the *expr.rs* of the parser.

Output: *unsat - the specification correctly defines the implementation.*

Program 42

Accessing tuple with unpacking:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Error, Integer, Map, Set, Text, SMT};

#[smt_impl]
fn pack() -> Integer {
    let (y1, y2) = (Integer::from(1), Integer::from(1));
    let x = Integer::from(0);
    y2.mul(x)
}

#[smt_spec(impls = pack)]
fn pack_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    pack_spec().eq(Integer::from(0))
}
```

SMT Output

```
; verification of impl-spec pair: pack ~> pack_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes ()
 ((Tuple_Integer_Integer
 (mk-Tuple_Integer_Integer
 (field_Tuple_Integer_Integer_1_ Int)
 (field_Tuple_Integer_Integer_2_ Int)))))

; Define user functions:
(define-fun-rec pack () Int
 (* 1 0)
)

(declare-fun pack_spec () Int)

; Define axioms:
(assert (= pack_spec 0))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (pack ) (pack_spec )))))
(assert (not (= pack pack_spec)))

(check-sat)
(exit)
```

Challenges: Previously, all the variables on the left-hand side of the tuple were bound to the whole tuple on the right-hand side. Now they are bound to the corresponding expression on the right hand side. The following change was done:

```
let e = self.registry.lookup_exp(exp).clone();
if let Expression::Pack {
    sort: _,
    elems: elems_pack,
} = e
{
    if elems_pack.len() != elems.len() {
        panic!(
            "pack slot number mismatch: expect {} | actual {}",
            elems_pack.len(),
            elems.len()
        );
    }
    for ((elem_decl, elem_sort), ex) in elems.iter().zip(tuple).zip(elems_pack) {
        self.bind_decl(elem_decl, elem_sort, ex);
    }
```

```
} else {
    panic!("expect a pack expression");
}
```

This inside exp.rs in the ir module.

Output: *unsat - the specification correctly defines the implementation.*

Program 43

Match expressions with tuples:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Cloak, Error, Integer, Map, Set, Text, SMT};

#[smt_type]
enum MyStruct {
    Unit,
    Struct(Integer, Integer),
    Record { a: Integer, b: Integer },
}
#[smt_impl]
fn pack() -> Integer {
    // inside a match we must have an enum expression
    match (
        MyStruct::Unit,
        MyStruct::Struct(Integer::from(1), Integer::from(2)),
        MyStruct::Record {
            a: Integer::from(3),
            b: Integer::from(4),
        },
    ) {
        (MyStruct::Unit, _, _) => Integer::from(0),
        (MyStruct::Struct(_, _), _, _) => Integer::from(1),
        (MyStruct::Record { a, b }, _, _) => a.add(b),
    }
}

#[smt_spec(impls = pack)]
fn pack_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn ax() -> Boolean {
    pack_spec().eq(Integer::from(1).sub(Integer::from(1)))
}
```

SMT Output

```
; verification of impl-spec pair: pack ~> pack_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((MyStruct (Record (record_MyStruct_Record_a_ Int)
↪  (record_MyStruct_Record_b_ Int)) (Struct (field_MyStruct_Struct_1_ Int)
↪  (field_MyStruct_Struct_2_ Int)) (Unit))))

; Define user functions:
(define-fun-rec pack () Int
```

```
12   (ite (and (is-Record Unit) (is-Record (Struct 1 2)) (is-Record (Record 3 4))) (+
     ↪  (record_MyStruct_Record_a_ Unit) (record_MyStruct_Record_b_ Unit)) (ite (and (is-Record
     ↪  Unit) (is-Record (Struct 1 2)) (is-Struct (Record 3 4))) (+ (record_MyStruct_Record_a_
     ↪  Unit) (record_MyStruct_Record_b_ Unit)) (ite (and (is-Record Unit) (is-Record (Struct 1
     ↪  2)) (is-Unit (Record 3 4))) (+ (record_MyStruct_Record_a_ Unit)
     ↪  (record_MyStruct_Record_b_ Unit)) (ite (and (is-Record Unit) (is-Struct (Struct 1 2))
     ↪  (is-Record (Record 3 4))) (+ (record_MyStruct_Record_a_ Unit) (record_MyStruct_Record_b_
     ↪  Unit)) (ite (and (is-Record Unit) (is-Struct (Struct 1 2)) (is-Struct (Record 3 4))) (+
     ↪  (record_MyStruct_Record_a_ Unit) (record_MyStruct_Record_b_ Unit)) (ite (and (is-Record
     ↪  Unit) (is-Struct (Struct 1 2)) (is-Unit (Record 3 4))) (+ (record_MyStruct_Record_a_
     ↪  Unit) (record_MyStruct_Record_b_ Unit)) (ite (and (is-Record Unit) (is-Unit (Struct 1
     ↪  2)) (is-Record (Record 3 4))) (+ (record_MyStruct_Record_a_ Unit)
     ↪  (record_MyStruct_Record_b_ Unit)) (ite (and (is-Record Unit) (is-Unit (Struct 1 2))
     ↪  (is-Struct (Record 3 4))) (+ (record_MyStruct_Record_a_ Unit) (record_MyStruct_Record_b_
     ↪  Unit)) (ite (and (is-Record Unit) (is-Unit (Struct 1 2)) (is-Unit (Record 3 4))) (+
     ↪  (record_MyStruct_Record_a_ Unit) (record_MyStruct_Record_b_ Unit)) (ite (and (is-Struct
     ↪  Unit) (is-Record (Struct 1 2)) (is-Record (Record 3 4))) 1 (ite (and (is-Struct Unit)
     ↪  (is-Record (Struct 1 2)) (is-Struct (Record 3 4))) 1 (ite (and (is-Struct Unit)
     ↪  (is-Record (Struct 1 2)) (is-Unit (Record 3 4))) 1 (ite (and (is-Struct Unit) (is-Struct
     ↪  (Struct 1 2)) (is-Record (Record 3 4))) 1 (ite (and (is-Struct Unit) (is-Struct (Struct
     ↪  1 2)) (is-Struct (Record 3 4))) 1 (ite (and (is-Struct Unit) (is-Struct (Struct 1 2))
     ↪  (is-Unit (Record 3 4))) 1 (ite (and (is-Struct Unit) (is-Unit (Struct 1 2)) (is-Record
     ↪  (Record 3 4))) 1 (ite (and (is-Struct Unit) (is-Unit (Struct 1 2)) (is-Struct (Record 3
     ↪  4))) 1 (ite (and (is-Struct Unit) (is-Unit (Struct 1 2)) (is-Unit (Record 3 4))) 1 (ite
     ↪  (and (is-Unit Unit) (is-Record (Struct 1 2)) (is-Record (Record 3 4))) 0 (ite (and
     ↪  (is-Unit Unit) (is-Record (Struct 1 2)) (is-Struct (Record 3 4))) 0 (ite (and (is-Unit
     ↪  Unit) (is-Record (Struct 1 2)) (is-Unit (Record 3 4))) 0 (ite (and (is-Unit Unit)
     ↪  (is-Struct (Struct 1 2)) (is-Record (Record 3 4))) 0 (ite (and (is-Unit Unit) (is-Struct
     ↪  (Struct 1 2)) (is-Struct (Record 3 4))) 0 (ite (and (is-Unit Unit) (is-Struct (Struct 1
     ↪  2)) (is-Unit (Record 3 4))) 0 (ite (and (is-Unit Unit) (is-Unit (Struct 1 2)) (is-Record
     ↪  (Record 3 4))) 0 (ite (and (is-Unit Unit) (is-Unit (Struct 1 2)) (is-Struct (Record 3
     ↪  4))) 0 0))))))))))))))))))))))))))))))
13   )
14
15   (declare-fun pack_spec () Int)
16
17   ; Define axioms:
18   (assert (= pack_spec (- 1 1)))
19
20   ; Prove the equivalence of the operational and denotational semantics:
21   ; (assert (forall () (=>  (= (pack ) (pack_spec )))))
22   (assert (not (= pack pack_spec)))
23
24   (check-sat)
25   (exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 44

Seq expression:

Rust Program

```rust
// testing Boolean
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{exists, forall, Boolean, Integer, Seq, SMT};

#[smt_impl]
fn _identity_(x1: Seq<Integer>) -> Seq<Integer> {
    let x2: Seq<Integer> = Seq::new(); // incomplete type error if explicit type not mentioned
    let a = x1.at_unchecked(Integer::from(0));
    let x3: Seq<Integer> = x2.append(a);
    x3
}

#[smt_spec(impls = _identity_)]
fn _identity_spec_(x1: Seq<Integer>) -> Seq<Integer> {
    unimplemented!()
}

#[smt_axiom]
fn _identity_axiom_(x: Seq<Integer>) -> Boolean {
    if *x.length().eq(Integer::from(1)) {
        _identity_spec_(x).eq(x)
    } else {
        Boolean::from(false)
    }
}
```

SMT Output

```smt
; verification of impl-spec pair: _identity_ ~> _identity_spec_
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(declare-const seq_0 (Seq Int))
(assert (= seq_0 (as seq.empty (Seq Int)))) ; seq.empty
(define-fun-rec _identity_ ((x1 (Seq Int))) (Seq Int)
 (seq.++ seq_0 (seq.unit (seq.nth x1 0)))
)

(declare-fun _identity_spec_ ((Seq Int)) (Seq Int))

; Define axioms:
(assert (forall ((x (Seq Int))) (ite (= (seq.len x) 1) (= (_identity_spec_ x) x) false)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x1 (Seq Int))) (=> (= x1 x1) (= (_identity_ x1) (_identity_spec_ x1)))))
(declare-const x1 (Seq Int))
(assert (= x1 x1))
(assert (not (= (_identity_ x1) (_identity_spec_ x1))))

(check-sat)
(exit)
```

Challenges: No issue. **Output:** *unsat - the specification correctly defines the implementation.*

Program 45

Testing quantifiers:

Rust Program

```rust
// this program makes no sense and is just for testing purposes of forall and exists
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{choose, exists, forall, Boolean, Integer, Seq, SMT};

#[smt_impl]
fn seq_min() -> Integer {
    Integer::from(1)
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom() -> Boolean {
    forall!(|x: Integer| x.mul(seq_min_spec()).eq(x))
}
```

SMT Output

```smt
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec seq_min () Int
 1
)

(declare-fun seq_min_spec () Int)

; Define axioms:
(assert (forall ((x_0 Int)) (= (* x_0 seq_min_spec) x_0)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (seq_min ) (seq_min_spec )))))
(assert (not (= seq_min seq_min_spec)))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 46

Testing quantifiers:

Rust Program

```rust
// this program makes no sense and is just for testing purposes of forall and exists
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min() -> Integer {
    Integer::from(1)
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom() -> Boolean {
    forall!(|x: Integer,
             y: Boolean,
             w: Rational,
             h: Text,
             o: Seq<Integer>,
             i: Set<Integer>,
             p: Map<Integer, Boolean>,
             q: Cloak<Integer>|
             x.mul(seq_min_spec()).eq(x))
}
```

SMT Output

```smt
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec seq_min () Int
 1
)

(declare-fun seq_min_spec () Int)

; Define axioms:
(assert (forall ((x_0 Int) (y_1 Bool) (w_2 Real) (h_3 String) (o_4 (Seq Int)) (i_5 (Set Int)) (p_6 (Array Int Bool)) (q_7 Int)) (= (* x_0 seq_min_spec) x_0)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (seq_min ) (seq_min_spec )))))
(assert (not (= seq_min seq_min_spec)))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 47

Testing quantifiers:

Rust Program

```rust
// x * -1 = x => x == 0
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min() -> Integer {
    Integer::from(0)
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom() -> Boolean {
    seq_min_spec().eq(choose!(|x: Integer| x.mul(Integer::from(-1)).eq(x)))
}
```

SMT Output

```smt
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec seq_min () Int
 0
)

(declare-fun seq_min_spec () Int)

; Define axioms:
(declare-const x_choose_0 (Int))
(assert (= (* x_choose_0 -1) x_choose_0))
(assert (= seq_min_spec x_choose_0))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (seq_min ) (seq_min_spec )))))
(assert (not (= seq_min seq_min_spec)))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 48

Testing quantifiers:

Rust Program

```rust
// x - x = 0
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min() -> Integer {
    Integer::from(0)
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec() -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom() -> Boolean {
    forall!(|x:Integer| x.sub(x).eq(seq_min_spec()))
}
```

SMT Output

```
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec seq_min () Int
 0
)

(declare-fun seq_min_spec () Int)

; Define axioms:
(assert (forall ((x_0 Int)) (= (- x_0 x_0) seq_min_spec)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (seq_min ) (seq_min_spec )))))
(assert (not (= seq_min seq_min_spec)))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 49

Testing quantifiers:

Rust Program

```rust
// for no x => x^2 < 0
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min() -> Boolean {
    Boolean::from(false)
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec() -> Boolean {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom() -> Boolean {
    exists!(|x: Integer| x.mul(x).lt(Integer::from(0))).eq(seq_min_spec())
}
```

SMT Output

```smt
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user functions:
(define-fun-rec seq_min () Bool
 false
)

(declare-fun seq_min_spec () Bool)

; Define axioms:
(assert (= (exists ((x_0 Int)) (< (* x_0 x_0) 0)) seq_min_spec))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall () (=>  (= (seq_min ) (seq_min_spec )))))
(assert (not (= seq_min seq_min_spec)))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 50

Testing quantifiers:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min(set: Set<Integer>) -> Integer {
    choose! (x in set => forall! (y in set => x.lt(y).or(x.eq(y))))
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec(set: Set<Integer>) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom(set: Set<Integer>) -> Boolean {
    exists!(x in set => seq_min_spec(set).gt(x)).not()
}
```

SMT Output

```
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

(declare-const set (Set Int))
; Define user functions:
(declare-const x_choose_1 (Int))
(assert (=> (select set x_choose_1) (forall ((y_2 Int)) (=> (select set y_2) (or (< x_choose_1 y_2) (= x_choose_1 y_2))))))
(define-fun-rec seq_min ((set (Set Int))) Int
 x_choose_1
)

(declare-fun seq_min_spec ((Set Int)) Int)

; Define axioms:
(assert (forall ((set (Set Int))) (not (exists ((x_1 Int)) (=> (select set x_1) (> (seq_min_spec set) x_1))))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((set (Set Int))) (=> (= set set) (= (seq_min set) (seq_min_spec set)))))
(assert (= set set))
(assert (not (= (seq_min set) (seq_min_spec set))))

(check-sat)
(exit)
```

Challenges: No issue - however, for this case we moved the declaration of the function parameters to the start of the smt script as some assertions which were defined outside of the function depended on these parameters.

Output: *unsat - the specification correctly defines the implementation.*

Program 51

Testing infer:

Rust Program

```rust
// testing infer
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min(set: Set<Integer>) -> Integer {
    let a = Integer::from(0);
    let b = a;
    let c = b;
    c
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec(set: Set<Integer>) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom(set: Set<Integer>) -> Boolean {
    seq_min_spec(set).eq(Integer::from(0))
}
```

SMT Output

```smt
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

(declare-const set (Set Int))
; Define user functions:
(define-fun-rec seq_min ((set (Set Int))) Int
 0
)

(declare-fun seq_min_spec ((Set Int)) Int)

; Define axioms:
(assert (forall ((set (Set Int))) (= (seq_min_spec set) 0)))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((set (Set Int))) (=> (= set set) (= (seq_min set) (seq_min_spec set)))))
(assert (= set set))
(assert (not (= (seq_min set) (seq_min_spec set))))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*

Program 52

Testing matches when it has one case and enums with variants the same name as the rusmart system type:

Rust Program

```rust
// testing Boolean(V)
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_type]
enum V {
    Boolean(Boolean),
}

#[smt_impl]
fn seq_min(v: V) -> Integer {
    match v {
        V::Boolean(b) => {
            if *b {
                Integer::from(1)
            } else {
                Integer::from(0)
            }
        }
    }
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec(v: V) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom(b: Boolean) -> Boolean {
    if *b {
        seq_min_spec(V::Boolean(b)).eq(Integer::from(1))
    } else {
        seq_min_spec(V::Boolean(b)).eq(Integer::from(0))
    }
}
```

SMT Output

```
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((V (Boolean (field_V_Boolean_1_ Bool)))))

(declare-const v V)
; Define user functions:
(define-fun-rec seq_min ((v V)) Int
 (ite (field_V_Boolean_1_ v) 1 0)
)

(declare-fun seq_min_spec (V) Int)

; Define axioms:
(assert (forall ((b Bool)) (ite b (= (seq_min_spec (Boolean b)) 1) (= (seq_min_spec (Boolean b)) 0))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((v V)) (=> (= v v) (= (seq_min v) (seq_min_spec v)))))
(assert (= v v))
(assert (not (= (seq_min v) (seq_min_spec v))))

(check-sat)
(exit)
```

Challenges: No issue - fixed the bug in the match translation.

Output: *unsat - the specification correctly defines the implementation.*

Program 53

Testing quantifiers over sequences:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min(s: Seq<Boolean>) -> Seq<Boolean> {
    let s2: Seq<Boolean> = s.append(Boolean::from(false));
    let s3: Seq<Boolean> = s2.append(Boolean::from(false));
    let s4: Seq<Boolean> = s3.append(Boolean::from(false));
    s4
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec(s: Seq<Boolean>) -> Seq<Boolean> {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom(s: Seq<Boolean>) -> Boolean {
    // seq_min_spec(s) here is a Seq<Integer> not a Seq<Boolean>
    // if we had seq_min_spec(s) as Seq<Integer> then i could refer to the index or element
    forall!(i in seq_min_spec(s) => seq_min_spec(s).at_unchecked(i).and(Boolean::from(true)).eq(Boolean::from(false)))
}
```

SMT Output

```smt
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

(declare-const s (Seq Bool))
; Define user functions:
(define-fun-rec seq_min ((s (Seq Bool))) (Seq Bool)
 (seq.++ (seq.++ (seq.++ s (seq.unit false)) (seq.unit false)) (seq.unit false))
)

(declare-fun seq_min_spec ((Seq Bool)) (Seq Bool))

; Define axioms:
(assert (forall ((s (Seq Bool))) (forall ((i_1 Int)) (= (and (seq.nth (seq_min_spec s) i_1) true) false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((s (Seq Bool))) (=> (= s s) (= (seq_min s) (seq_min_spec s)))))
(assert (= s s))
(assert (not (= (seq_min s) (seq_min_spec s))))

(check-sat)
(exit)
```

Challenges: The issues and assumptions are mentioned as comments in the rusmart program.

Output: *sat - the specification does not correctly define the implementation.* Because for example an empty sequence gives can satisfy the axiom but is not equal to the implementation.

Program 54

Testing local blocks:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_impl]
fn seq_min(s: Seq<Boolean>) -> Seq<Boolean> {
    let x = {
        let s2: Seq<Boolean> = s.append(Boolean::from(false));
        let s3: Seq<Boolean> = s2.append(Boolean::from(false));
        let s4: Seq<Boolean> = s3.append(Boolean::from(false));
        s4
    };
    // let s2: Seq<Integer> = Seq::new(); // uncommenting this will cause an error: conflicting variable names
    // here testing that the vars in the local are not outside
    let s2: Seq<Integer> = Seq::new();
    x
}

#[smt_spec(impls = seq_min)]
fn seq_min_spec(s: Seq<Boolean>) -> Seq<Boolean> {
    unimplemented!()
}

#[smt_axiom]
fn seq_min_axiom(s: Seq<Boolean>) -> Boolean {
    // seq_min_spec(s) here is a Seq<Integer> not a Seq<Boolean>
    // if we had seq_min_spec(s) as Seq<Integer> then i could refer to the index or element
    forall!(i in seq_min_spec(s) => seq_min_spec(s).at_unchecked(i).and(Boolean::from(true)).eq(Boolean::from(false)))
}
```

SMT Output

```smt2
; verification of impl-spec pair: seq_min ~> seq_min_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

(declare-const s (Seq Bool))
; Define user functions:
(define-fun-rec seq_min ((s (Seq Bool))) (Seq Bool)
 (seq.++ (seq.++ (seq.++ s (seq.unit false)) (seq.unit false)) (seq.unit false))
)

(declare-fun seq_min_spec ((Seq Bool)) (Seq Bool))

; Define axioms:
(assert (forall ((s (Seq Bool))) (forall ((i_1 Int)) (= (and (seq.nth (seq_min_spec s) i_1) true) false))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((s (Seq Bool))) (=> (= s s) (= (seq_min s) (seq_min_spec s)))))
(assert (= s s))
(assert (not (= (seq_min s) (seq_min_spec s))))

(check-sat)
(exit)
```

Challenges: No issue; also checked the variables in the scope, after exiting the block the variables are removed.

Output: *sat - the specification does not correctly define the implementation.* Because for example an empty sequence gives can satisfy the axiom but is not equal to the implementation.

Program 55

Checking whether a method can have the same name as a standalone function or will it cause a duplicate error in the database:

Rust Program

```rust
use rusmart_smt_remark_derive::{smt_axiom, smt_impl, smt_spec, smt_type};
use rusmart_smt_stdlib::{
    choose, exists, forall, Boolean, Cloak, Error, Integer, Map, Rational, Seq, Set, Text, SMT,
};

#[smt_type]
struct MyInteger(Integer);

#[smt_impl(method = my_add)]
fn my_add(x: MyInteger, y: MyInteger) -> Integer {
    x.0.add(y.0)
}

// we can add as many methods we want for a type (like inside an impl block)
#[smt_impl(method = my_add2)]
fn my_add2(x: MyInteger, y: MyInteger) -> Integer {
    x.0.add(y.0)
}

// adding the following will cause an error because the method is already defined
// #[smt_impl(method = my_add)]
// fn my_add3(x: MyInteger, y: MyInteger) -> Integer {
//     x.0.add(y.0)
// }

#[smt_spec(impls = my_add)]
fn my_add_spec(x: MyInteger, y: MyInteger) -> Integer {
    unimplemented!()
}

#[smt_axiom]
fn my_add_axiom(x: MyInteger, y: MyInteger) -> Boolean {
    my_add_spec(x, y).eq(x.my_add(y))
}
```

SMT Output

```smt2
; verification of impl-spec pair: my_add ~> my_add_spec
(set-option :print-success false)
(set-option :produce-models true)
(set-option :smt.string_solver z3str3)
(set-option :smt.string_solver seq)

; Define user-defined types:
(declare-datatypes () ((MyInteger (mk-MyInteger (field_MyInteger_1_ Int)))))

(declare-const x MyInteger)
(declare-const y MyInteger)
; Define user functions:
(define-fun-rec my_add ((x MyInteger) (y MyInteger)) Int
 (+ (field_MyInteger_1_ x) (field_MyInteger_1_ y))
)

(declare-fun my_add_spec (MyInteger MyInteger) Int)

; Define axioms:
(assert (forall ((x MyInteger) (y MyInteger)) (= (my_add_spec x y) (my_add x y))))

; Prove the equivalence of the operational and denotational semantics:
; (assert (forall ((x MyInteger) (y MyInteger)) (=> (and (= x x) (= y y)) (= (my_add x y) (my_add_spec x y)))))
(assert (= x x))
(assert (= y y))
(assert (not (= (my_add x y) (my_add_spec x y))))

(check-sat)
(exit)
```

Challenges: No issue.

Output: *unsat - the specification correctly defines the implementation.*